Chapter 11

IMPLEMENTATION OF EULER OPERATORS

This chapter provides information on the implementation of Euler operators. It describes several layers of procedures needed for this purpose, ranging from level of storage allocation for the data structures to Euler operators themselves.

11.1 OVERVIEW

The actual implementation of Euler operators is not a difficult task. Nevertheless, considerable care must be used to make sure that all cases are covered (such as empty loops and "circular" edges). The algorithms of GWB will not at all be afraid of creating the most complicated of cases.

We shall follow a "bottom-up" approach in describing the implementation. Hence we start from the lowest levels of interest for the purposes of this book, and build additional levels on top of them. This leads to a very economical design in terms of the code size (if not necessarily speed).

11.2 A STORAGE ALLOCATOR

The most primitive functionality needed for Euler operators consists of storage allocation and deallocation. We shall here build our own allocator on the top of the ordinary UNIX utilities. In practice, this allocator should be replaced by a more rigorous version that itself keeps track of the available storage and avoids the costly UNIX memory allocation procedure `malloc`.

```
unsigned nodesize[] =
{
        sizeof(Solid), sizeof(Face), sizeof(Loop),
        sizeof(HalfEdge), sizeof(Edge), sizeof(Vertex), 0,
};

Node    *new(what, where)
int     what;
Node    *where;
{
        Node    *node;
        char    *malloc();

        node = (Node *) malloc(nodesize[what]);
        switch(what)
        {
                case SOLID:
                        addlist(SOLID, node, NIL);
                        node->s.sfaces = (Face *) NIL;
                        node->s.sedges = (Edge *) NIL;
                        node->s.sverts = (Vertex *) NIL;
                        break;
                case FACE:
                        addlist(FACE, node, where);
                        node->f.floops = (Loop *) NIL;
                        node->f.flout = (Loop *) NIL;
                        break;
                case LOOP:
                        addlist(LOOP, node, where);
                        break;
                case EDGE:
                        addlist(EDGE, node, where);
                        break;
                case VERTEX:
                        addlist(VERTEX, node, where);
                        node->v.vedge = (Vertex *) NIL;
                        break;
                default:
                        break;
        }
        return(node);
}
```

Program 11.1 Storage allocation.

A design goal of the allocator is to simplify the implementation of Euler operators by pushing the handling of many of the pointers of the half-edge data structure into the allocator and other very low-level routines. This is accomplished by giving the allocator information on where the new node is to be inserted.

The allocator **new** of Program 11.1 returns a pointer to a new node of the data structure; good C programming style requires that a type conversion is specified when calling the allocator. The type of the node is selected with argument *what*, which must be one of the literals **SOLID**, **FACE**, **LOOP**, **HALFEDGE**, **EDGE**, or **VERTEX**. Another argument *where* must point to a parent node of the new node as follows:

1. *what* = **FACE**, **EDGE**, or **VERTEX**: *where* points to the parent solid.

2. *what* = **LOOP**: *where* points to the parent face.

In these cases **new** performs the necessary manipulations to link the new node with its parent. Otherwise *where* has no effect and may be **NIL**. Array *nodesize* contains the sizes of the various nodes.

Analogously to the storage allocator **new**, the deallocator procedure **del(what, which, where)** removes the node *which* from the data structure, and disconnects it from the list of the children of the parent node *where* if *what* = **SOLID**, **FACE**, **EDGE**, **LOOP**, or **VERTEX**. All our algorithms will assume that the data in a deleted node remains valid until the next call of **new**. We leave the implementation of **del** to the reader.

11.2.1 Linked List Procedures

The actual insertion of faces, loops, edges, and vertices into the proper lists takes place with the procedure **addlist** given in Program 11.2. It adds the node pointed to by *which* into the linked list of nodes owned by *where*. As a special case, solids are simply linked to the list of all solids; in this case, *where* has no effect. The type of *which* is again given in terms of *what* and must be one of the literals **SOLID**, **FACE**, **LOOP**, **EDGE**, or **VERTEX**.

To unlink children from their parents, the corresponding inverse procedure **dellist(what, which, where)** is used. Specifically, the node *which* of type *what* (**SOLID**, **FACE**, **LOOP**, **EDGE**, **VERTEX**) will be removed from the respective linked list of the parent node *which*. Again, solids are an exception. Obviously, **dellist** would be used in the implementation of **del** in an analogous way to **addlist** and **new**. The implementation of **dellist** is not elaborated here.

```
void  addlist(what, which, where)
int   what;
Node  *which, *where;
{
    switch(what)
    {
        case SOLID:
            which->s.nexts = firsts;
            which->s.prevs = (Solid *) NIL;
            if(firsts)
                firsts->prevs = (Solid *) which;
            firsts = (Solid *) which;
            break;
        case FACE:
            which->f.nextf = where->s.sfaces;
            which->f.prevf = (Face *) NIL;
            if(where->s.sfaces)
                where->s.sfaces->prevf = (Face *) which;
            where->s.sfaces = (Face *) which;
            which->f.fsolid = (Solid *) where;
            break;
        case LOOP:
            which->l.nextl = where->f.floops;
            which->l.prevl = (Loop *) NIL;
            if(where->f.floops)
                where->f.floops->prevl = (Loop *) which;
            where->f.floops = (Loop *) which;
            which->l.lface = (Face *) where;
            break;
        case EDGE:
            which->e.nexte = where->s.sedges;
            which->e.preve = (Edge *) NIL;
            if(where->s.sedges)
                where->s.sedges->preve = (Edge *) which;
            where->s.sedges = (Edge *) which;
            break;
        case VERTEX:
            which->v.nextv = where->s.sverts;
            which->v.prevv = (Vertex *) NIL;
            if(where->s.sverts)
                where->s.sverts->prevv = (Vertex *) which;
            where->s.sverts = (Vertex *) which;
            break;
    }
}
```

Program 11.2 Linked list manipulation.

11.2.2 Halfedge Procedures

In the implementation of the allocator, the actual initialization of halfedges and edges is performed by procedure **addhe**. Referring to Program 11.3, the procedure allocates a new halfedge, links it into loop l at the front of an existing halfedge *where*, and sets the other pointers appropriately. Note that **addhe** is written so as to handle the special cases of an empty original loop (**where->edg** = NIL) or no loop at all (**where** = NIL).

The inverse procedure **delhe** of Program 11.4 performs the opposite actions. Note in particular how it creates an "empty" loop in the case that **he->edg** $\neq$ NIL and **he->nxt** = **he**.

Algorithms of GWB will do the bulk of their work with Euler operators. Occasionally, however, they will also use some of the functions described above. For instance, the sectioning algorithm of Chapter 14 will use **new**, **addlist**, and **dellist** to actually divide the sectioned solid.

11.3 LOW-LEVEL EULER OPERATORS

Euler operators will be available in GWB in two versions. In the lower layer, the operators are specified by means of pointers to adjacent nodes of the half-edge data structure, and they can work without any searching of the data structure. In the upper layer, the operators will be parameterized in terms of face and vertex identifiers, and they will have to search for the relevant nodes of the data structure.

Based on the storage allocator and related procedures, this section describes the implementation of low-level Euler operators, while the high-level operators will be the topic of the next section. Because of space limitations, it is not possible to give a fully detailed implementation of each operator. Instead, we give three examples that should give a good idea of how the remaining ones should be written.

11.3.1 MVFS

Logically the first Euler operator we need is **mvfs**, the initialization Euler operator. Based on the procedures developed in the preceding sections, the concise implementation of Program 11.5 can readily be given.

Referring to the program, the procedure allocates one each of nodes *Solid*, *Face*, *Loop*, and *Vertex* with **new**, and one *HalfEdge* with **addhe**. Note how **new** will take care of linking child nodes properly into their parents. A new halfedge is properly initialized to represent the "empty" loop consisting of one vertex but no edges at all. The reader is encouraged to sketch with pencil and paper the resulting data structure.

```
HalfEdge        *addhe(e, v, where, sign)
Edge            *e;
Vertex          *v;
HalfEdge        *where;
int             sign;
{
        HalfEdge        *he;

        if(where->edg == NIL)
        {
                he = where;
        }
        else
        {
                he = (HalfEdge *) new(HALFEDGE, NIL);
                where->prv->nxt = he;
                he->prv = where->prv;
                where->prv = he;
                he->nxt = where;
        }

        he->edg = e;
        he->vtx = v;
        he->wloop = where->wloop;
        if(sign == PLUS)
                e->he1 = he;
        else    e->he2 = he;

        return(he);
}
```

Program 11.3 Halfedge manipulation.


```
HalfEdge        *delhe(he)
HalfEdge        *he;
{
        if(he->edg == NIL)
        {
                del(HALFEDGE, he, NIL);
                return(NIL);
        }
        else if(he->nxt == he)
        {
                he->edg = NIL;
                return(he);
        }
        else
        {
                he->prv->nxt = he->nxt;
                he->nxt->prv = he->prv;
                del(HALFEDGE, he, NIL);
                return(he->prv);
        }
}
```

Program 11.4 Halfedge deletion.

```
Solid   *mvfs(s, f, v, x, y, z)
Id      s, f, v;
float   x, y, z;
{
        Solid           *newsolid;
        Face            *newface;
        Vertex          *newvertex;
        HalfEdge        *newhe;
        Loop            *newloop;

        newsolid = (Solid *) new(SOLID, NIL);
        newface = (Face *) new(FACE, newsolid);
        newloop = (Loop *) new(LOOP, newface);
        newvertex = (Vertex *) new(VERTEX, newsolid);
        newhe = (HalfEdge *) new(HALFEDGE, NIL);

        newsolid->solidno = s;
        newface->faceno = f;
        newface->flout = newloop;
        newloop->ledg = newhe;
        newhe->wloop = newloop;
        newhe->nxt = newhe->prv = newhe;
        newhe->vtx = newvertex;
        newhe->edg = NIL;
        newvertex->vertexno = v;
        newvertex->vcoord[0] = x;
        newvertex->vcoord[1] = y;
        newvertex->vcoord[2] = z;
        newvertex->vcoord[3] = 1.0;

        return(newsolid);
}
```

Program 11.5 The mvfs operator.

11.3.2 LMEV

As the next example, we pick the operator lmev, the low-level vertex-splitting operator. Recall that we expect lmev to be capable of dealing with all special cases of empty loops, "strut" edges, and so on. It turns out that this can be implemented in a relatively natural and concise fashion.

Referring to the code of Program 11.6, if the arguments *he1* and *he2* of the procedure differ, lmev will divide the cycle of halfedges of vertex he1->vtx in two cycles so that *he1* is the first halfedge in one cycle, and *he2* in the other. The identifier *vn*, the coordinates x y z and the cycle starting at *he1* are assigned to a new vertex. A new edge joins he1->vtx and the new vertex. In the case that *he1* = *he2*, a "strut" edge that appears twice at a loop is added at front of *he1*.

Nicely enough, the halfedge manipulation procedures allow this specification to be implemented without any special code for different cases. The procedure first allocates and initializes the new vertex and edge nodes. As halfedges from *he1* (inclusive) to *he2* (noninclusive) will be assigned to a new vertex, the while-loop updates their vertex pointers. Note how macro mate is again used to select the next halfedge of the cycle. Finally, new halfedges are inserted, and the cycle pointers of vertices updated. Note that the new edge will be oriented towards the old vertex. Again, the reader is encouraged to check the effects of the code in various cases (e.g., when applied to the halfedge created by mvfs).

11.3.3 LMEF

The next example is the code for lmef, the low-level face splitting operator. Dually to lmev, if *he1* $\neq$ *he2*, lmef is expected to divide the loop of *he1* and *he2* into two loops with a new edge from he1->vtx to he2->vtx. Halfedge *he2* remains in the "old" loop, while *he1* is moved to the new loop that becomes the outer boundary of a new face. In the special case that *he1* = *he2*, a "circular" face with just one edge is created.

The code implementing this is given in Program 11.7. It is written so as to stress the dualism of MEF and MEV, and the reader is urged to compare Programs 11.6 and 11.7 closely. This code splits the original loop by first inserting the two halfedges as usually, and then "swapping" their tails. This has the nice feature that no special code for "circular" new faces (i.e., case *he1* = *he2*) is needed.

```
void            lmev(he1, he2, v, x, y, z)
HalfEdge        *he1, *he2;
Id              v;
float           x, y, z;
{
        HalfEdge        *he;
        Vertex          *newvertex;
        Edge            *newedge;

        newedge = (Edge *) new(EDGE, he1->wloop->lface->fsolid);
        newvertex = (Vertex *) new(VERTEX,
                he1->wloop->lface->fsolid);
        newvertex->vcoord[0] = x;
        newvertex->vcoord[1] = y;
        newvertex->vcoord[2] = z;
        newvertex->vcoord[3] = 1.0;
        newvertex->vertexno = v;

        he = he1;
        while(he != he2)
        {
                he->vtx = newvertex;
                he = mate(he)->nxt;
        }

        addhe(newedge, newvertex, he2, PLUS);
        addhe(newedge, he2->vtx, he1, MINUS);

        newvertex->vedge = he2->prv;
        he2->vtx->vedge = he2;
}
```

Program 11.6 The lmev operator.


```
Face            *lmef(he1, he2, f)
HalfEdge        *he1, *he2;
Id              f;
{
        Face            *newface;
        Loop            *newloop;
        Edge            *newedge;
        HalfEdge        *he, *nhe1, *nhe2, *temp;

        newface = (Face *)new(FACE, he1->wloop->lface->fsolid);
        newloop = (Loop *)new(LOOP, newface);
        newedge = (Edge *)new(EDGE, he1->wloop->lface->fsolid);
        newface->faceno = f;
        newface->flout = newloop;

        he = he1;
        while(he != he2)
        {
                he->wloop = newloop;
                he = he->nxt;
        }

        nhe1 = addhe(newedge, he2->vtx, he1, MINUS);
        nhe2 = addhe(newedge, he1->vtx, he2, PLUS);

        nhe1->prv->nxt = nhe2;
        nhe2->prv->nxt = nhe1;
        temp = nhe1->prv;
        nhe1->prv = nhe2->prv;
        nhe2->prv = temp;

        newloop->ledg = nhe1;
        he2->wloop->ledg = nhe2;

        return(newface);
}
```

Program 11.7 The lmef operator.

11.3.4 LKEMR

The final example is the code of lkemr, the low-level operator for splitting a loop into two components.

The actual splitting of the loop is implemented in Program 11.8 in the most straightforward fashion. After a new loop node has been allocated, the two halfedges *h1* and *h2* corresponding with the edge to be removed are manipulated so as to effectively leave them in two distinct components, of which the one associated with *h2* becomes the new loop. This manipulation has the virtue that delhe can be used to actually delete *h1* and *h2*, and the various special cases of empty loops are eliminated.

Operators that remove edges (lkev, lkef, lkemr) must be particularly carefully written. In this case, it might happen that the loop nodes refer to a halfedge just deleted, so the *ledg*-pointers of the loops must be updated. Similarly, the vertices of *h1* and *h2* may refer back to *h1* and *h2* through the *vedge*-pointer.

11.4 HIGH-LEVEL EULER OPERATORS

The implementation of high-level Euler operators is based on their low-level counterparts and auxiliary procedures that search the half-edge data structure in order to find all the required pointers. In the simple implementation presented below, all searches are implemented as sequential scans. When dealing with solids with more than, say, 100 faces, an implementation allowing direct access to faces based on hashing should be used.

Referring to Program 11.9, the procedure getsolid will locate the solid with the identifier *sn* from the list of all solids. If the procedure is successful, it returns a pointer to the solid; otherwise, an empty pointer is returned.

Similarly, the procedure fface locates the face of *s* with identifier *fn*, and returns a pointer to it. After getting our hands on a face *f*, procedure fhe seeks a halfedge from *vn1* to *vn2*. Again, if these procedures cannot find what they seek, they return NIL.

With the scanning procedures, the implementation of high-level Euler operators is straightforward. For instance, the code for the high-level operator mev is given in Program 11.10. After finding the desired pointers, the procedure simply calls the corresponding low-level operator lmev to do the job. Most of the code is actually spent in catching errors in solid, face, or vertex identifiers. The procedure returns a code indicating the success or failure of the operation requested. It is good coding practice to give all high-level Euler operators this capability.


```
void            lkemr(h1, h2)
HalfEdge        *h1, *h2;
{
        register HalfEdge       *h3, *h4;
        Loop                    *nl;
        Loop                    *ol;
        Edge                    *killedge;

        ol = h1->wloop;
        nl = (Loop *) new(LOOP, ol->lface);
        killedge = h1->edg;

        h3 = h1->nxt;
        h1->nxt = h2->nxt;
        h2->nxt->prv = h1;
        h2->nxt = h3;
        h3->prv = h2;

        h4 = h2;
        do
        {
                h4->wloop = nl;
        }
        while((h4 = h4->nxt) != h2);

        ol->ledg = h3 = delhe(h1);
        nl->ledg = h4 = delhe(h2);

        h3->vtx->vedge = (h3->edg) ? h3 : (HalfEdge *) NIL;
        h4->vtx->vedge = (h4->edg) ? h4 : (HalfEdge *) NIL;

        del(EDGE, killedge, ol->lface->fsolid);
}
```

Program 11.8 The lkemr operator.

```
Solid       *getsolid(sn)
Id          sn;
{
            Solid   *s;
            for(s = firsts; s != NIL; s = s->nexts)
                    if(s->solidno == sn)
                            return(s);
            return(NIL);
}

Face        *fface(s, fn)
Solid       *s;
Id          fn;
{
            Face    *f;
            for(f = s->sfaces; f != NIL; f = f->nextf)
                    if(f->faceno == fn)
                            return(f);
            return(NIL);
}

HalfEdge    *fhe(f, vn1, vn2)
Face        *f;
Id          vn1, vn2;
{
            Loop            *l;
            HalfEdge        *h;
            for(l = f->floops; l != NIL; l = l->nextl)
            {
                    h = l->ledg;
                    do
                    {
                            if(h->vtx->vertexno == vn1 &&
                               h->nxt->vtx->vertexno == vn2)
                                    return(h);
                    }
                    while((h = h->nxt) != l->ledg);
            }
            return(NIL);
}
```

Program 11.9 Scanning procedures.


```
int     mev(s, f1, f2, v1, v2, v3, v4, x, y, z)
Id      s, f1, f2, v1, v2, v3, v4;
float   x, y, z;
{
        Solid           *oldsolid;
        Face            *oldface1, *oldface2;
        HalfEdge        *he1, *he2;

        if((oldsolid = getsolid(s)) == NIL)
        {
                fprintf(stderr, "mev: solid %d not found\n", s);
                return(ERROR);
        }
        if((oldface1 = fface(oldsolid, f1)) == NIL)
        {
                fprintf(stderr,
                        "mev: face %d not found in solid %d\n", f1, s);
                return(ERROR);
        }
        if((oldface2 = fface(oldsolid, f2)) == NIL)
        {
                fprintf(stderr,
                        "mev: face %d not found in solid %d\n", f2, s);
                return(ERROR);
        }
        if((he1 = fhe(oldface1, v1, v2)) == NIL)
        {
                fprintf(stderr,
                        "mev: edge %d-%d not found in face %d\n",
                                v1, v2, f1);
                return(ERROR);
        }
        if((he2 = fhe(oldface2, v1, v3)) == NIL)
        {
                fprintf(stderr,
                        "mev: edge %d-%d not found in face %d\n",
                                v1, v3, f2);
                return(ERROR);
        }
        lmev(he1, he2, v4, x, y, z);
        return(SUCCESS);
}
```

Program 11.10 The mev operator.

11.5 C CALLING SEQUENCES

This section specifies the details of the programming interface to all Euler operators and related procedures as used in Part Two by giving summaries of their calling sequences from C programming language. In some cases the effects are depicted in terms of partial plane models; in all such illustrations, arrowheads are used to denote the orientations of halfedges and edges. Orientations of all faces are clockwise.

11.5.1 Low-Level Euler Operators

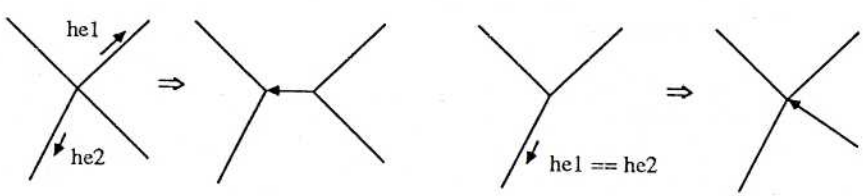
```
Solid    *mvfs(sn, f, v, x, y, z)
Id       sn, f, v;
float    x, y, z;
```

Operator `mvfs` creates a new solid named *sn* that consists of one face *f* and one vertex *v* with coordinates *x*, *y*, and *z*. It returns a pointer to the new solid created. Note that the operator `mvfs` is the only one that has just one level of implementation: there are no separate low-level and high-level operators.

```
void     lkvfs(s)
Solid    *s;
```

Operator `lkvfs` is the inverse of `mvfs`, and removes the solid pointed at by *s*. The solid is assumed to consist of a single face and vertex only.

```
void     lmev(he1, he2, v, x, y, z)
HalfEdge *he1, *he2;
Id       v;
float    x, y, z;
```

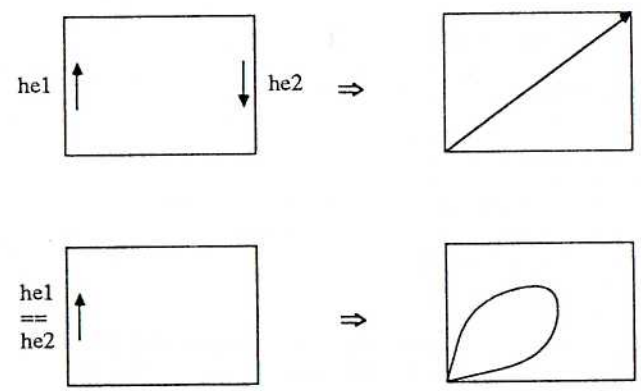


Operator `lmev` "splits" the vertex pointed at by *he1* and *he2*, and adds a new edge between the resulting two vertices. The identifier *v* and the coordinates *x*, *y*, and *z* are assigned to the new vertex. The case that *he1* and *he2* are identical is allowed; in this case, the new vertex and edge are added into the face of *he1*. The new edge is oriented from the new vertex to the old one.

```
void     lkev(he1, he2)
HalfEdge *he1, *he2;
```

Operator `lkev` is the inverse of `lmev`. It removes the edge pointed at by both *he1* and *he2*, and "joins" the two vertices `he1->vtx` and `he2->vtx` which must be distinct (but can, of course, correspond with geometrically identical points). Vertex `he1->vtx` is removed.

```
Face     *lmef(he1, he2, f)
HalfEdge *he1, *he2;
Id       f;
```



Operator `lmef` adds a new edge between halfedges *he1* and *he2*, and "splits" their common face into two faces such that *he1* will occur in the new face *f*, and *he2* remains in the old face. The new edge is oriented from `he1->vtx` to `he2->vtx`. Halfedges *he1* and *he2* must belong to the same loop (i.e., `he1->wloop == he2->wloop`). They may be equal, in which case a "circular"

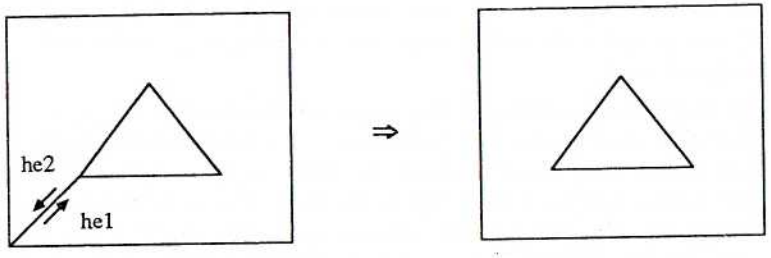
face with just one edge is created. A pointer to the new face is returned.

Observe that `lmef` does not attempt to classify the loops of the divided face into the faces resulting from the division, i.e., all other loops of the old face remain in it. The auxiliary procedure `ringmv` described at the end of this section can be used to move the loops to the new face, if desired.

```
void      lkef(he1, he2)
HalfEdge *he1, *he2;
```

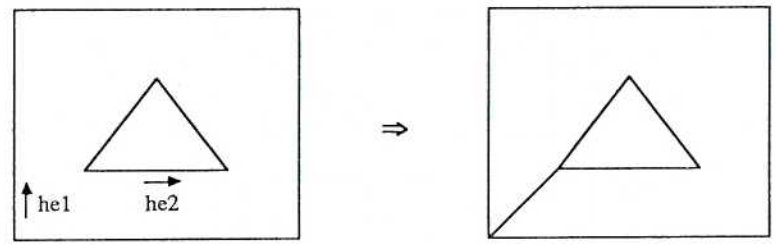
Operator `lkef` is the inverse of `lmef`. It removes the edge of *he1* and *he2*, and "joins" the two adjacent faces by merging their loops. The face `he2->wloop->lface` is removed. `lkef` is applicable to the halves of an edge that occurs in two distinct faces. That is, it is assumed that `he1->edg == he2->edg` and `he1->wloop->lface != he2->wloop->lface`.

```
void      lkemr(he1, he2)
HalfEdge *he1, *he2;
```



Operator `lkemr` removes the edge of *he1* and *he2*, and divides their common loop into two components (i.e., it creates a new "ring"). If the original loop was "outer," the component of `he1->vtx` becomes the new "outer" loop. (If this default is inappropriate, you can simply swap the arguments of `lkemr`, or use `lringmv` to make the desired loop "outer".) It is assumed that `he1->edg == he2->edg` and `he1->wloop == he2->wloop`.


```
void      lmekr(he1, he2)
HalfEdge *he1, *he2;
```



Operator `lmekr` is the inverse of `lkemr`. It inserts a new edge between the vertices of *he1* and *he2*, and merges the corresponding loops into one loop (i.e., removes a ring). The operator assumes that `he1->wloop != he1->wloop`, i.e., *he1* and *he2* belong to two distinct loops, and that `he1->wloop->lface == he2->wloop->lface`, i.e., they occur in a single face.

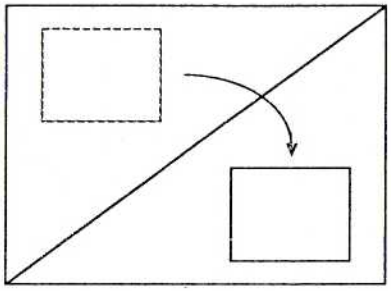
```
void      lkfmrh(fac1, fac2)
Face      *fac1, *fac2;
```

Operator `lkfmrh` merges two faces *fac1* and *fac2* by making the loop of the latter a ring into the former. Face *fac2* is hence removed. It is assumed that *fac2* is simple, i.e., has just one loop.

```
void      lmfkrh(l, f)
Loop      *l;
Id        f;
```

Operator `lmfkrh` is the inverse of `lkfmrh`. It makes the loop *l* the outer loop of a new face *f*. It is assumed that *l* is an inner loop of its parent face.

```
void    lringmv(l, tofac, inout)
Loop    *l;
Face    *tofac;
int     inout;
```



Procedure `lringmv` moves the loop *l* from its parent face to face *tofac*. If *inout* = 0 *l* becomes an "inner" loop, and otherwise the outer loop of *tofac*. If `l->lface == tofac`, `lringmv` has no effect except for assigning the inner vs. outer information. Note that `lringmv` is an addendum to `lmef`, and not an Euler operator.

11.5.2 High-Level Euler Operators

All high-level operators take a list of vertex, face, and solid identifiers as their input. Having no names themselves, edges and halfedges are identified through their vertices. The notation *v1* → *v2* is used below to denote the edge between vertices *v1* and *v2*.

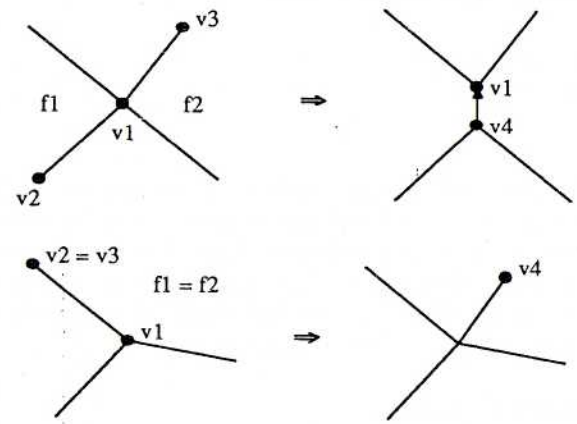
The parameterization of high-level operators can be quite mechanically derived from the low-level ones by replacing pointers to halfedges by a triple (*f*, *v1*, *v2*), where *v1* → *v2* is an edge appearing in the face *f*. Because this leads to quite complicated parameter lists, alternative parameterizations for some operators will be given.

```
void    kvfs(s)
Id      s;
```

Operator `kvfs` is the inverse of `mvfs`, and removes the solid *s*.


```
int     mev(s, f1, f2, v1, v2, v3, v4, x, y, z)
Id      s, f1, f2, v1, v2, v3, v4;
float   x, y, z;

int     smev(s, f1, v1, v4, x, y, z)
Id      s, f1, v1, v4;
float   x, y, z;
```



Operator `mev` divides the cycle of edges around the vertex *v1* so that all edges from *v1* → *v2* (inclusive) to *v1* → *v3* (exclusive) will become adjacent to a new vertex *v4*. The vertices *v1* and *v4* are joined with a new edge. Coordinates *x*, *y*, and *z* are assigned to *v4*.

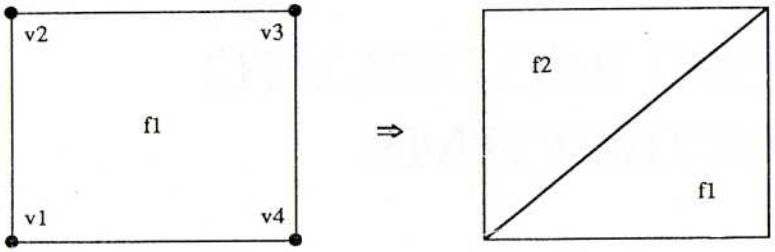
Similarly to the corresponding low-level operator, various special cases are possible. Of particular use is the "line-drawing" case that *f1* = *f2* and *v2* = *v3*. In this case, a new edge to a new vertex will be added within the face. We shall call such "dangling" edges *struts*. This case occurs so frequently that we include a "convenience" procedure `smev` that performs this operation. The procedure assumes that *v1* appears just once in *f1*; hence, the argument *v2* can be left out.

```
int     kev(s, f, v1, v2)
Id      s, f, v1, v2;
```

Operator `kev` is the exact inverse of `mev`. It removes the edge *v1* → *v2* from face *f* by merging *v2* with *v1*.

```
int     mef(s, f1, v1, v2, v3, v4, f2)
Id      s, f1, f2, v1, v2, v3, v4;

int     smef(s, f1, v1, v3, f2)
Id      s, f1, f2, v1, v3;
```

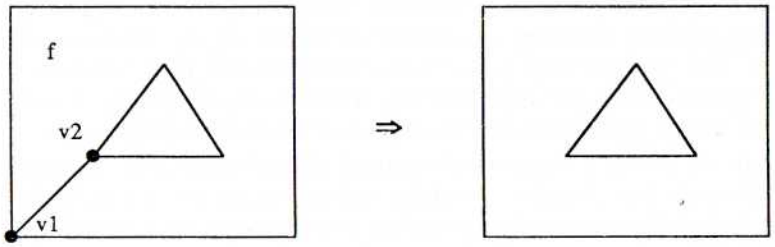


Operator mef connects the vertices *v1* and *v3* of face *f1* with a new edge, and creates a new face *f2*. Similarly to smev, we include the convenience procedure smef that leaves the arguments *v1* and *v4* out. It should be applied only if *v1* and *v3* are known to occur just once in the face.

```
int     kef(s, f, v1, v2)
Id      s, f, v1, v2;
```

Operator kef removes the edge *v1* → *v2*, and joins the two faces separated by it.

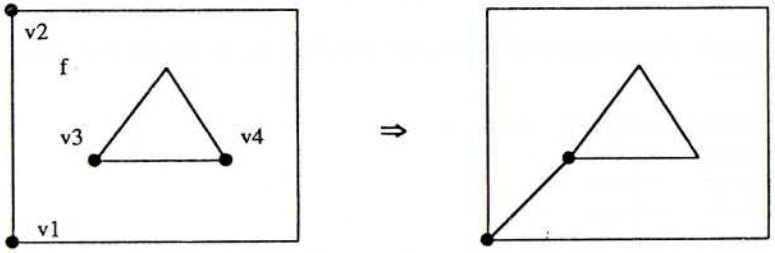
```
int     kemr(s, f, v1, v2)
Id      s, f, v1, v2;
```



Operator kemr removes the edge *v1* → *v2*, and divides the loop that contained it into two parts.


```
int     mekr(s, f, v1, v2, v3, v4)
Id      s, f, v1, v2, v3, v4;

int     smekr(s, f, v1, v3)
Id      s, f, v1, v3;
```



Operator mekr, the inverse of kemr, joins two vertices *v1* and *v3* from distinct loops of face *f* with an edge. The convenience procedure can be used if *v1* and *v3* are known to occur just once in their loops.

```
int     kfmrh(s, f1, f2)
Id      s, f1, f2;
```

Operator kfmrh "merges" two faces *f1* and *f2* by making the latter an interior loop of the former. Face *f2* is removed.

```
int     mfkrh(s, f1, vi, v2, f2)
Id      s, f1, vi, v2, f2;
```

Operator mfkrh makes the interior loop of face *f1* which contains the edge *v1* → *v2* the boundary of a new face *f2*. It is the inverse of kfmrh.

```
int     ringmv(s, f1, f2, v1, v2, inout)
Solid   *s;
Id      f1, f2. v1, v2, inout;
```

The procedure ringmv moves a ring from face *f1* to face *f2*. The ring moved is the one that has the edge *v1* → *v2*. An empty loop is denoted by *v2* = 0. If *inout* = 1 the moved loop becomes the "outer" loop of *f2*. The procedure is the high-level counterpart of lringmv.

PROBLEMS

11.1. Implement the list removal procedure **dellist** that has the following C calling sequence:

```
void   dellist(what, which, where)
int    what;
Node   *which;
Node   *where;
```

11.2. Implement the storage deallocator **del** that has the following C calling sequence:

```
void   del(what, which, where)
int    what;
Node   *which;
Node   *where;
```

(You will need the list deletion procedure **dellist** of Problem 11.1.)

11.3. Implement the operators **lkev** and **lkef** obeying the calling sequences of Section 11.5.2.

11.4. Implement the operator **lmekr** according to its functional specification in Section 11.5.2.

11.5. Implement the operators **lkfmrh** and **lmfkrh** as defined in Section 11.5.2.